



Manual de Acesso, Configuração e Funções

Revisão completa do código-fonte

12/07/2026 · Next.js 16

01 · CONTEXTO

Visão geral

O Rentech Web é o site institucional e o sistema de gestão interna da Rentech — locadora de equipamentos audiovisuais (LED, videowall, TV, som e luz). Um único projeto serve o site público e o painel administrativo (RH, financeiro, estoque, automações).

CAMADA	TECNOLOGIA
Framework	Next.js 16.2.6 (App Router, Server Actions)
UI	React 19.2.4 + Tailwind CSS v4
Linguagem	TypeScript
Banco / Auth / Storage	Supabase (Postgres + Auth + Storage)
E-mail	Nodemailer (SMTP)
Geração de PDF	pdf-lib
OCR de documentos	AWS Textract
Assinatura eletrônica	Autentique (API GraphQL)
WhatsApp	Z-API
Hospedagem	Vercel (com Vercel Cron)
Analytics	Vercel Analytics

Nota técnica — este projeto usa o Next.js 16, que renomeou convenções de versões anteriores. Exemplo já em uso: o middleware não se chama `middleware.ts / middleware()`, e sim `proxy.ts / proxy()` (seção 5). O arquivo `AGENTS.md` do projeto já alerta qualquer pessoa (ou IA) que for mexer no código.

Como rodar o projeto localmente

Pré-requisitos: Node.js instalado e as variáveis de ambiente configuradas (seção 3).

```
npm install      # instala as dependências
npm run dev      # inicia o servidor de desenvolvimento (http://localhost:3000)
npm run build    # gera o build de produção
npm run start    # roda o build de produção localmente
npm run lint     # roda o ESLint
```

O projeto é publicado automaticamente na Vercel (seção 10).

Variáveis de ambiente

Ficam em `.env.local` (não versionado no Git). **Nenhum valor de segredo é reproduzido aqui** — apenas os nomes das chaves e para que servem. Os valores reais devem ser obtidos com quem administra cada serviço.

Supabase

VARIÁVEL	USO
<code>NEXT_PUBLIC_SUPABASE_URL</code>	URL do projeto Supabase (pública)
<code>NEXT_PUBLIC_SUPABASE_ANON_KEY</code>	Chave pública/anônima (usada no navegador)
<code>SUPABASE_SERVICE_ROLE_KEY</code>	Chave de serviço — acesso total ao banco, ignora as regras de segurança (RLS). Uso exclusivo no servidor.

E-mail (SMTP)

VARIÁVEL	USO
<code>SMTP_HOST</code> , <code>SMTP_PORT</code> , <code>SMTP_USER</code> , <code>SMTP_PASS</code>	Credenciais do servidor de e-mail para orçamentos e notificações de OP

Site

VARIÁVEL	USO
<code>NEXT_PUBLIC_SITE_URL</code>	URL base, usada para montar links em e-mails (confirmação de OP, recibo)

Portal de Downloads

VARIÁVEL	USO
<code>DOWNLOADS_PASSWORD</code>	Senha única que libera o acesso à página pública <code>/downloads</code>

Autentique (assinatura eletrônica)

VARIÁVEL	USO
<code>AUTENTIQUE_API_TOKEN</code>	Token de acesso à API GraphQL da Autentique

AWS (OCR de documentos)

VARIÁVEL	USO
<code>AWS_REGION</code> , <code>AWS_ACCESS_KEY_ID</code> , <code>AWS_SECRET_ACCESS_KEY</code>	Credenciais AWS para o Textract ler PDFs contábeis no módulo Financeiro do RH

Z-API (WhatsApp) e Cron

VARIÁVEL	USO
<code>ZAPI_INSTANCE</code> , <code>ZAPI_TOKEN</code> , <code>API_CLIENT_TOKEN</code>	Credenciais da instância Z-API (envio/recebimento de WhatsApp)
<code>CRON_SECRET</code>	Protege a rota <code>/api/cron/motor</code> — só a Vercel Cron deve conhecê-lo
<code>ZAPI_WEBHOOK_SECRET</code>	Enviado como <code>?token=</code> na URL do webhook de ponto, configurado no painel Z-API

Em produção — as mesmas variáveis precisam ser cadastradas em *Project Settings* → *Environment Variables* na Vercel; o `.env.local` só vale localmente.

04 • ORGANIZAÇÃO

Estrutura de pastas

<code>app/</code>	
<code>├─ page.tsx</code>	→ Home institucional
<code>├─ layout.tsx</code>	→ Layout raiz (Navbar + metadata)
<code>├─ login/</code>	→ Login administrativo
<code>├─ freelance/</code>	→ Cadastro público de freelancers
<code>├─ downloads/</code>	→ Portal de downloads (protegido por senha)
<code>├─ recibo/[id]/</code>	→ Assinatura pública de recibo de OP
<code>├─ simulador/</code>	→ Simuladores comerciais (videowall, tela, grid, curvatura)
<code>├─ admin/</code>	→ Painel administrativo (seção 7)
<code>├─ api/</code>	→ Rotas de API e webhooks (seção 8)
<code>├─ lib/</code>	→ Supabase, Autentique, Z-API, PDF, automações
<code>├─ actions.ts</code>	→ Server actions globais (orçamento, downloads, auditoria)
<code>└─ imgs/</code>	→ Imagens usadas como módulo (logo)
<code>components/</code>	→ Navbar, Carrossel de vídeo, Contato
<code>public/</code>	→ Logos, imagens de cases, vídeos
<code>proxy.ts</code>	→ Middleware (proteção de rota <code>/admin/op</code>)
<code>vercel.json</code>	→ Configuração da Vercel (Cron Job)

Autenticação e controle de acesso

O login é feito via **Supabase Auth** (e-mail/senha) em `/login` . Ao autenticar, o sistema grava o cookie `sb-access-token` e registra o acesso no log de auditoria.

Existem **duas camadas de proteção**, e elas não são iguais:

1 · `proxy.ts` (edge, antes da página carregar) → protege só `/admin/op/*`

2 · `app/admin/layout.tsx` (navegador, depois da página carregar) → protege o restante do `/admin/**`

A camada 2 é mais fraca que a camada 1 porque o código já chegou ao navegador antes da checagem acontecer (ver seção 11).

Níveis de permissão

O texto de permissão cadastrado no perfil do usuário é normalizado (função `normalizarPermissao`) em uma destas categorias:

CATEGORIA	RECONHECIDA A PARTIR DE (CONTÉM)
ADMINISTRADOR	"ADMIN", "DIR" (diretoria), "GEREN" (gerência)
ADMINISTRATIVO	—
FINANCEIRO	"FINAN"
OPERACIONAL	"OPER"
ESTOQUE	"ESTOQ"
EDITOR	"EDIT"
USUARIO	padrão, quando nada mais se encaixa

A gestão de quem tem qual permissão é feita em **Permissões** (seção 7.3), lendo a tabela `setores_permissao` .

Site público — páginas e funções

PÁGINA	FUNÇÃO
/	Home institucional: hero, carrossel de vídeos, cases e formulário de orçamento. Conteúdo vem do Supabase (SiteConfig), editável em Conteúdo (7.5). Envia e-mail via enviarOrcamento .
/login	Login administrativo (Supabase Auth).
/freelance	Cadastro público de freelancers (nome, CPF, PIX, níveis LED/videowall/TV/áudio/luz), com consentimento LGPD.
/downloads	Portal de arquivos protegido por senha única. Lista arquivos do Supabase Storage.
/recibo/[id]	Assinatura pública de recibo de OP — assinatura por canvas, salva no banco. Acessada via link por e-mail.
/simulador (+ /videowall , /tela , /grid , /curvatura)	Simuladores técnicos/comerciais para dimensionar telas de LED, videowall, TV e calcular curvatura/grid.

Área administrativa (/admin)

Requer login. O layout comum (`app/admin/layout.tsx`) confere a sessão, registra "ACESSO AO SISTEMA" no log de auditoria (uma vez por sessão) e atualiza `ultimo_acesso` do usuário.

7.1 Dashboard e permissões	7.2 Estoque	7.3 Permissões
7.4 Log de auditoria	7.5 Conteúdo (CMS)	7.6 Downloads
7.7 Freelancers	7.8 Integrações	7.9 Agendamentos
7.10 Ordens de Pagamento	7.11 RH / Folha	

7.1 • Dashboard e permissões

`/admin` — hub central com a lista de módulos disponíveis, cujo acesso varia conforme a permissão do usuário logado (seção 5).

7.2 • Estoque

`/admin/estoque` — cadastro e gestão de equipamentos (categorias e itens) da locadora.

7.3 • Permissões de usuários

`/admin/permissoes` — gestão de usuários, papéis e setores (tabela `setores_permissao`). É aqui que se define quem é ADMINISTRADOR, FINANCEIRO, OPERACIONAL etc.

7.4 • Log de auditoria

`/admin/log` — histórico de ações do sistema (`logs_auditoria`): logins, acessos, disparos de automação, mudanças de status de OP. Quase toda ação sensível grava um registro aqui via `registrarLogAuditoria`.

7.5 • Conteúdo do site (CMS)

`/admin/conteudo` — editor tipo CMS para a home pública: textos do hero, imagens, WhatsApp e e-mail de contato. Grava em `SiteConfig`.

7.6 • Downloads (admin)

`/admin/downloads` — upload e remoção dos arquivos exibidos no portal público
`/downloads`.

7.7 • Freelancers (admin)

`/admin/freelance` — revisão dos cadastros enviados pelo formulário público
`/freelance`.

7.8 • Integrações

`/admin/integracao` — status das integrações externas: cadastro de parceiros/bancos/assinatura (`folha_integracoes`), verificação se o token da Autentique está configurado (com estatísticas de uso) e status da conexão Z-API.

Nunca expõe segredos, apenas indicadores.

7.9 • Agendamentos e disparos (automações WhatsApp)

`/admin/agendamentos` — CRUD de automações de disparo (`folha_automacoes`), do tipo:

- `CRON` disparadas automaticamente em horário programado, verificado a cada 5 min por `/api/cron/motor` (seção 8);
- `WEBHOOK` disparadas por evento.

Kill switch — cada automação tem um campo `ativo`: é a chave geral que o motor de cron verifica antes de disparar qualquer coisa. Desligar aqui impede o disparo mesmo que esteja agendada.

7.10 • OP — Ordens de Pagamento

TELA	FUNÇÃO
<code>/admin/op</code>	Lista e gerencia as OPs; dispara e-mail de notificação (<code>dispararEmailOP</code>).
<code>/admin/op/nova</code>	Cria uma nova OP.
<code>/admin/op/responsavel</code>	Lista de OPs filtrada para o responsável designado.
<code>/admin/op/financeiro</code>	Visão financeira das OPs.

OP criada



e-mail ao responsável

confirmação em `/api/baixar-op`assinatura em `/recibo/[id]`

7.11 • RH — Recursos Humanos / Folha

`/admin/rh` — o **módulo mais complexo do sistema**, cobrindo folha de pagamento, ponto, benefícios e documentos.

TELA	FUNÇÃO
Funcionário	Cadastro (CRUD) dos funcionários.
Holerite	Fechamento/reabertura de folha em lote; envio para assinatura eletrônica (Autentique).
Ponto	Importação de batidas/abonos, livro-razão via bot de WhatsApp, fila de justificativas pendentes de aprovação. Inclui "Separar Holerites" — divide um PDF contábil em um arquivo por funcionário (pdf-lib/pdf.js, no navegador).
Benefícios	Catálogo e atribuição de VR/VT/benefícios flash, incluindo geração de cargas.
Parâmetros	Feriados e regras/catálogos usados nos cálculos da folha.
Documentos	Cofre de documentos dos funcionários — categorias, upload, controle de validade.
Assinaturas	Painel Autentique: listar, consultar status, reenviar, baixar assinado, atualizar tudo.
Relatórios	Relatórios (ex.: grade de benefícios).
Financeiro	Montagem do lote de salários: OCR via AWS Textract em PDFs contábeis, preparação do arquivo bancário.

Pendência conhecida — o envio efetivo ao banco no módulo Financeiro ainda é um placeholder — depende de credenciais/certificado ainda não configurados (seção 12).

O bot de WhatsApp registra pedidos de ponto/justificativa/abono num livro-razão apenas de inserção, mas **nunca aprova nada sozinho** — toda justificativa precisa ser aprovada manualmente pelo RH na tela Ponto.

Rotas de API e Webhooks

ROTA	MÉTODO	FUNÇÃO	PROTEÇÃO
/api/baixar-op	GET	Link por e-mail: confirma pagamento da OP (status → PAGO) e mostra página de sucesso.	—
/api/videos	GET	Lista vídeos em public/videos para o carrossel.	—
/api/cron/motor	GET	Alvo do Vercel Cron (a cada 5 min). Dispara automações CRON ativas cujo horário/dia bate com o horário atual (fuso Brasil).	Authorization: Bearer CRON_SECRET
/api/webhooks/autentique	POST / GET	Recebe notificações de envio/visualização/assinatura/rejeição e atualiza status no banco. GET é health check.	Formato de payload validado
/api/webhooks/zapi-ponto	POST / GET	Recebe mensagens de WhatsApp do bot de ponto (PONTO / JUSTIFICAR / ABONAR) e responde.	?token=ZAPI_WEBHOOK_SECRET

Apenas /api/cron/motor está registrada em vercel.json como cron job; as demais são chamadas pelo site ou configuradas como webhook nos painéis Autentique/Z-API.

Integrações externas

SERVIÇO	PARA QUE SERVE	ONDE CONFIGURAR
Supabase	Banco de dados, autenticação, storage	Painel Supabase → <code>NEXT_PUBLIC_SUPABASE_*</code> , <code>SUPABASE_SERVICE_ROLE_KEY</code>
SMTP	E-mails de orçamento e OP	Provedor de e-mail → <code>SMTP_*</code>
AWS Textract	OCR de PDFs contábeis	Console AWS (IAM) → <code>AWS_*</code>
Autentique	Assinatura eletrônica de holerites/recibos	Painel Autentique → <code>AUTENTIQUE_API_TOKEN</code> + webhook para <code>/api/webhooks/autentique</code>
Z-API	WhatsApp (bot de ponto e automações)	Painel Z-API → <code>ZAPI_*</code> , <code>API_CLIENT_TOKEN</code> + webhook para <code>/api/webhooks/zapi-ponto</code>
Vercel Cron	Dispara o motor de automações a cada 5 min	<code>vercel.json</code> + <code>CRON_SECRET</code>
Vercel Analytics	Métricas de uso	Automático via pacote

Deploy e infraestrutura (Vercel)

- Hospedado na **Vercel**. Cada deploy precisa das mesmas variáveis da seção 3 cadastradas em *Project Settings* → *Environment Variables*.
- **Cron job**: configurado em `vercel.json`, chama `GET /api/cron/motor` a cada 5 minutos.
- **Git LFS**: dois arquivos grandes versionados via LFS — `public/cases/corporativo.gif` e `public/videos/video2.mp4`. É preciso ter o Git LFS instalado para clonar corretamente.
- **Limite de upload em Server Actions**: 10 MB, configurado em `next.config.ts` (necessário para uploads de PDF no RH e nos downloads).

Segurança — observações e recomendações

1 • Proteção de rota assimétrica

Apenas `/admin/op/*` é protegida no middleware (`proxy.ts`), que roda antes da página carregar. As demais seções do admin dependem só de uma checagem **no navegador**, depois da página já ter sido enviada. Recomenda-se ampliar o matcher do `proxy.ts` para `/admin/:path*` se quiser reforçar.

2 • Segredo do webhook de ponto na URL

`/api/webhooks/zapi-ponto` usa `?token=` como única proteção, já que a Z-API não assina requisições. Aceitável, mas o token pode ficar exposto em logs de acesso — tratar como segredo e trocar se vaziar.

3 • Chave de serviço do Supabase

`SUPABASE_SERVICE_ROLE_KEY` é a credencial mais sensível — acesso irrestrito ao banco, ignora RLS. Usada só em código de servidor (`supabaseAdmin()`), o que está correto; nunca deve aparecer em código de navegador.

Nenhum valor de segredo foi incluído neste manual — apenas nomes de variáveis e finalidade.

Pontos de atenção / pendências encontradas

- **README.md** ainda é o texto padrão do `create-next-app` — não documenta o projeto. Este manual supre essa lacuna (versão Markdown em `docs/MANUAL.md`).
- **Envio bancário** no módulo Financeiro do RH está implementado como placeholder — depende de credenciais/certificado ainda não configurados.
- `components/Contato.jsx` parece um formulário de exemplo, não conectado (simula envio com 1.5s de espera) — o formulário real da Home usa outra lógica.
- `.vscode/launch.json` aponta para a porta 8080, mas o Next.js roda por padrão na 3000 — provavelmente configuração antiga.
- Ícones padrão do Next.js (`file.svg` , `globe.svg` , `next.svg` , `window.svg` , `vercel.svg`) seguem em `public/` aparentemente sem uso — candidatos a limpeza.

Glossário

TERMO	SIGNIFICADO
OP	Ordem de Pagamento
RH	Recursos Humanos (módulo de folha de pagamento)
RLS	Row Level Security — regras de segurança por linha do Postgres/Supabase
OCR	Reconhecimento óptico de caracteres (leitura automática de texto em PDF/imagem)
CMS	Sistema de gestão de conteúdo (aqui, o editor da Home em <code>/admin/conteudo</code>)
Kill switch	Campo <code>ativo</code> que liga/desliga uma automação sem precisar apagá-la
LGPD	Lei Geral de Proteção de Dados (consentimento no formulário de freelancer)
Webhook	URL que um serviço externo chama automaticamente quando algo acontece
Server Action	Função que roda no servidor, chamada direto de um formulário/componente React
Middleware / <code>proxy.ts</code>	Código que roda antes da página, podendo bloquear/redirecionar o acesso

Manual gerado a partir de revisão completa do código-fonte do projeto **rentech-web** em 12/07/2026. Reflete o estado do código nessa data — recomenda-se atualizar sempre que módulos forem adicionados, removidos ou alterados. Versão Markdown mantida em `docs/MANUAL.md`, no próprio repositório.